CS3244: Final Progress Report

Project Title: "Totally Accurate" Sentiment Analysis

Group Number: 20

National University of Singapore

Group Members: anonymised

Mentor's Name: anonymised

Date of Submission: 19 April 2026

I. Introduction

Sarcasm is a common form of figurative language in which the speaker's intended meaning is the opposite of, or at least at odds with, its literal content. In text-based communication, this creates a significant challenge for automated sentiment analysis. The phrase "Oh great, another Monday" for example, reads as positive to a naive classifier but yet carries clear negative intent. This hidden meaning, unlike more explicitly straight forward terms like “disgusting”, “wonderful”, has a degree of ambiguity based on the context in which the sarcastic phrase was deployed in.

Sarcasm expresses negative sentiments using positive words, allowing it to easily confuse sentiment analysis models (Šandor & Bagić Babac, 2024). Failing to account for this can have downstream consequences across a range of NLP applications, from chatbots misreading user frustration as engagement, to content moderation systems failing to flag sarcastic harassment that bypasses keyword filters, to product review systems miscategorising customer dissatisfaction as praise.

Prior Work

Sarcasm detection research has evolved from early rule-based and lexical methods through machine learning classifiers to deep learning architectures (Verma et al., 2021). With the highest-performing models relying on Transformers, recurrent neural networks, and feature engineering (Potamias et al., 2020).

Early approaches relied on lexical and pragmatic cues. Features such as sentiment-bearing words, punctuation patterns, and part-of-speech distributions. Word embedding-based features paired with SVM classifiers were shown to be effective by emphasising the mismatch between sentiment-bearing words and overall sentence polarity as a frequent marker of sarcasm (Joshi et al., 2016). A significant advance came with the introduction of context-aware models: CASCADE integrated user profiling and discourse modelling with a CNN-based textual model, achieving state of the art results on a large-scale Reddit corpus (Hazarika et al., 2018).

More recently, transformer-based models have dominated benchmarks, with a BERT encoder combined with a recurrent convolutional network producing strong results on sarcasm and irony detection tasks (Potamias et al., 2020). It was further demonstrated that the same sentence could yield different sarcasm classifications depending on the surrounding dialogue history, underscoring the importance of contextual modeling (Pant & Dadu, 2020).

Limitations of Prior Work

Despite these advances, several limitations remain. Many deep learning and LSTM-based methods rely predominantly on sentence-level cues, neglecting broader contextual signals that are often necessary for sarcasm interpretation (Šandor & Bagić Babac, 2024). Sarcasm is also highly domain-dependent, appearing differently in political discourse, product reviews, or social media. This means that models trained on one corpus generalise poorly to other domains (Šandor & Bagić Babac, 2024).

Dataset construction introduces further problems, as self-annotated corpora inherit the noise and biases of the platform from which they are drawn, and models risk learning

platform-specific writing style rather than sarcasm itself (Khodak et al., 2017). At the systems level, most studies do not evaluate transformer models against each other in terms of accuracy versus efficiency, and existing models lack robust multi-turn conversation modelling (Raiyani et al., 2025).

Research Gap and Project Objective

These limitations point to a clear gap in the literature: most existing work either pursues maximum in-domain accuracy at the cost of generalisability, or deploys computationally expensive architectures without systematic comparison against simpler baselines. It remains unclear how much of the performance gain from transformer models is attributable to contextual understanding versus the sheer scale of pre-training.

This project addresses that gap by constructing a structured benchmark comparing classical ML approaches: Logistic Regression, Random Forest, and XGBoost against a transformer-based model on the SARC 1.0 dataset (Khodak et al., 2017). The aim is to empirically assess how much complexity in the models is warranted for sarcasm detection

Beyond academic interest, the practical stakes are meaningful: sarcasm can reverse the intended polarity of a sentiment classifier, chatbots risk misinterpreting sarcastic responses as positive engagement, and in mental health monitoring, sarcasm is often used to mask distress such that failing to detect it can result in missed warning signs (Šandor & Bagić Babac, 2024).

Machine Learning Approach

The machine learning concepts from this course map directly onto the challenge of sarcasm detection. Text representation via TF-IDF transforms raw comments into high-dimensional sparse feature vectors suitable for linear classification. Logistic regression serves as an interpretable linear baseline, where L1 and L2 regularization are applied to control overfitting in this high-dimensional space, directly applying the bias-variance tradeoff concepts covered in class. Ensemble methods are then introduced: Random Forest exploits bagging to reduce variance, while XGBoost applies boosting to iteratively correct residual errors, both drawing on ensemble learning principles from the course. Dimensionality reduction via TruncatedSVD is applied to make the sparse TF-IDF space tractable for gradient boosting.

Finally, a transformer-based model leverages pre-trained contextual embeddings to capture semantic contrast between a comment and its parent comment, a capability that bag-of-words approaches fundamentally cannot represent (Pant & Dadu, 2020). Together, these models form a progression from simple linear classifiers to deep contextual architectures, allowing us to make an empirical assessment of how much complexity is warranted for this task.

Dataset Description

The dataset chosen to evaluate the baseline model is the Self-Annotated Reddit Corpus (SARC) 1.0 sarcasm dataset, which contains 1.3 million comments from the Internet commentary website Reddit, with each comment being labelled as ‘sarcastic’ or ‘non-sarcastic’. Each instance of the dataset contains the comment text as the main feature, with secondary features such as author, date posted, upvotes, downvotes. This dataset provides a large corpus of comments that allows us to train our models on. The features have

a good variety of comments ranging from various subreddits. Such variety allows our models to encompass comments and texts from genres like gaming to technical discussions. While our approach does not account for these genres in our evaluation, ultimately the large quantity improves the efficacy of our testing.

Key Objective

Creating a framework to understand contextual meaning is thus key for this project. It enables the model to better detect sarcasm through examining surrounding phrases or sentences in which comments were made. An effective model to detect sarcasm can help strengthen traditional sentiment analysis models. For instance, traditional sentiment analysis models would categorise a sentence like "Yeah sure, that was such a genius idea 🙄" as positive because the word "genius" implies something positive. The usage of an emoji however, nuances the categorisation as it implies a level of mocking. Failing to capture the nuance of such phrases could cause spillover issues for chatbot systems like ChatGPT, Claude AI, content moderation systems, etc in detecting the true meaning behind the prompt or message of a user.

The goal of this project is thus to build and evaluate ML models that can distinguish between sarcastic and non-sarcastic comments, in order to improve automatic text understanding, make chatbot, content moderation systems more accurate.

II. Methodology & Approach

Pre-processing

The SARC 1.0 dataset is compiled from self-annotated Reddit comments (Khodak et al., 2017), where sarcasm labels are assigned by the authors themselves rather than independent annotators. While this mechanism enables the corpus to scale to over 1.3 million instances, it means the data inherits all the noise of raw user-generated web content, making pre-processing a necessary prerequisite to modeling. Four categories of issues require addressing: data validity (11 null entries that would cause undefined behaviour in tokenisers), structural noise (hyperlinks, mentions, and hashtags that inflate the vocabulary without adding predictive signal), representational inconsistency (irregular capitalisation causing semantically identical words to be treated as distinct features), and contextual completeness (the parent_comment requiring its own cleaning pass). Critically, pre-processing here is not solely about noise removal, it is equally about preserving the right signals, as sarcasm is sensitive to subtle linguistic cues such as punctuation.. To transform this into a model-ready format, we developed a four-stage pipeline in the projectText notebook designed to balance noise reduction with the preservation of subtle sarcastic signals. We first pruned the metadata to focus on high-signal features, which involved making deliberate decisions to discard certain columns.

The author field was removed to prevent the model from overfitting to specific users due to high cardinality, while temporal data and raw upvote/downvote counts were discarded as redundant. Our EDA justified this approach by showing that even the aggregated score was a weak predictor, as both sarcastic and non-sarcastic classes shared a nearly identical median of 2 with heavily overlapping distributions. On the other hand, the subreddit was retained as a vital contextual signal because sarcasm rates varied drastically between communities, ranging from 64.5% in worldnews to 39.9% in AskReddit.

The technical execution of this strategy began with a basic_clean function to handle null or empty entries, resulting in the removal of 11 invalid rows from the training set. This was followed by a normalize_text function that handles lowercasing and utilizes regular expressions to strip URLs, mentions, and hashtags. Crucially, we produced two variants: text_normalized, which preserves punctuation to help transformer models leverage structural cues, and text_no_punct for classical models to prevent the vocabulary from becoming overly fragmented.

We also integrated emoji.demojize to convert icons like 🙄 into text descriptors. While the SARC dataset itself has no emojis, this step acts as a safeguard to ensure our pipeline remains robust for emoji-heavy cross-corpus evaluations. Furthermore, because sarcasm often relies on a contrast with previous statements (Hazarika et al.,2018), we processed the parent_comment through the same cleaning sequence. This preprocessing pipeline ensures that both comment and contextual information are consistently structured and usable across models. Missing parent comments are mapped to empty strings rather than removed, allowing all training instances to retain a valid contextual field.

This design produces clean and flexible inputs that can be directly used for classical approaches such as TF-IDF vectorisation or tokenised for transformer-based models, ensuring consistency across different modelling strategies.

Baseline Model

Logistic regression was chosen as our baseline model due to its simplicity, interpretability and well-established effectiveness on high-dimensional sparse text data. We thus aim to use this logistic regression model to establish a strong linear baseline before moving to more complex models, as the performance gap reveals how much additional complexity is actually warranted.

Text was converted to TF-IDF vectors (we use a maximum of 50,000 features, unigrams + bigrams, min_df = 5). Although our preprocessing pipeline uses BERT tokenization, we do not use raw token IDs as features for logistic regression. BERT's vocabulary assigns each subword to an arbitrary integer index with no ordinal meaning. This means that "great" being token 2307 and "green" being token 2308 (for example) implies no relationship between them. If the raw token IDs are used, the integer itself would be the feature value, so the model would be trying to learn from numbers that are essentially random.

TF-IDF resolves this by instead assigns each dimension to a specific word or bigram, where the feature value is not a simple binary 1/0 for word presence, but a continuous score reflecting both how frequently the word appears in that specific comment and how rare it is across the entire corpus. For example, the word "obviously" appearing three times in a short comment gets a much higher value than "the" appearing three times as the word "the" would usually appear significantly more often throughout the entire corpus. This gives logistic regression features that it can meaningfully assign weights to, and makes the model directly interpretable: each learned coefficient reflects how strongly a word predicts sarcasm.
In the tuning process, we swept over L1 and L2 regularization with $C \in \{0.01, 0.1, 0.5, 1.0, 5.0, 10.0\}$. At very low C (very strong regularization), the model underfits: L1 with C = 0.001 used only 3 of 50,000 features, producing a validation F1-score of just 0.30. Performance improved steadily as C increased, peaking at L2, C = 1.0 (validation F1-score: 0.7089).

Beyond this, performance plateaued or slightly declined, indicating the onset of mild overfitting.

We also ran ablations on feature configuration. Bigrams provided a meaningful lift over unigrams alone (+3.0 percentage points on test F1 score), confirming that sarcastic phrases like “oh great” and “yeah right” are inherently multi-word constructs that unigrams would be unable to consider. Including the parent comment by concatenating it with the comment text hurt performance (-2.6 percentage points). This is likely because TF-IDF flattens the concatenation into a single bag of words, so the model cannot capture the contrast between the reply and its context, hence adding the parent comment simply adds noise and dilutes the signals

Tree-Based Models

To explore whether non-linear models and additional engineered features could improve over the linear baseline, we trained Random Forest and XGBoost on the same TF-IDF 50,000 features + subreddit target encoding (each subreddit mapped to its training-set sarcasm rate, Laplace-smoothed) + six engineered features including punctuation counts (exclamation marks, question marks, ellipses), ALL-CAPS ratio, comment length (word and character count). The subreddit encoding captures community-level sarcasm tendencies, while the punctuation and length features are motivated by the intuition that sarcasm is often accompanied by exaggerated stylistic markers and tends to be shorter and punchier than sincere comments

Random Forest was tuned over {100, 200, 300} estimators and {50, None} max depth. Shallow trees (depth = 50) consistently underperformed unconstrained trees, with val F1 jumping from ~0.655 to ~0.708 when depth was unconstrained, suggesting the model required considerable depth to capture meaningful patterns. For XGBoost, recognising that the raw 50,000-dimensional sparse TF-IDF space is structurally difficult for tree-based splitting, we first applied TruncatedSVD to reduce it to 300 dense dimensions before concatenating the engineered features, yielding a 308-dimensional dense matrix. We then swept over max depth $\in \{4, 6, 8\}$, learning rate $\in \{0.05, 0.1\}$, and column subsampling ratio $\in \{0.7, 0.9\}$, with early stopping on validation log-loss (n_estimators = 1000). The best config (depth=8, lr=0.1) stopped early at round 599, indicating genuine convergence.

Multi-Layer Perceptron Model

The model’s architecture was refined through three iterations to determine the better feature set for sarcasm detection:

1. **Child Comment Only:** Initial training focused solely on the response text, which often lacked sufficient context to identify sarcastic intent.
2. **Child + Parent Comment:** Performance improved by concatenating the parent comment, allowing the model to capture relational shifts in tone.
3. **Child + Parent + Subreddit Embedding:** This final iteration yielded the best results. Integrating community-specific embeddings provided the necessary social context to distinguish between sarcastic and literal discourse across different subreddits.

The chosen architecture is a hybrid model combining a DistilBERT transformer and a subreddit embedding layer, forming an Multi-Layer Perceptron (MLP) classifier. This

classifier consists of two hidden layers designed to improve reasoning capabilities, with each layer incorporating Batch Normalisation to stabilize the vector distributions, ReLU activation functions to introduce non-linearity, and Dropout for regularisation.

Sarcasm is deeply contextual, it depends on the relationship between a comment and the post or comment it is replying to. The self-attention mechanism in transformer models like DistilBERT is well-suited to capturing these relationships. The model's text inputs concatenate both the child comment and its parent using a [SEP] separator token, allowing the transformer to process both texts and capture how tone shifts relative to the parent. Dynamic padding was implemented via a custom collate function, padding each batch to its longest sequence rather than the globally defined max length. This meaningfully reduces memory usage and speeds up training, especially for shorter batches.

DistilBERT was chosen over a full BERT model as it retains approximately 97% of its performance while being 40% smaller and 60% faster. (Haziqa Sajid, 2024) This was a practical necessity given the computational resource constraints, particularly for large datasets, such as the 1.1 million training samples processed here. Additionally, only the last 3 transformer layers are unfrozen for fine-tuning to reduce computational cost and prevent overfitting, allowing the model to adapt to the Reddit dataset while preserving general pre-trained vocabulary.

The inclusion of a Subreddit Embedding Layer addresses possible contextual gaps found in prior sarcasm detection research (Potamias et al., 2020). Sarcasm is highly subreddit-specific; a statement that is sarcastic in a political forum may be literal in a hobbyist group, making it a meaningful novel addition. To manage the vocabulary size of over 16,000 subreddits, only subreddits appearing at least 100 times in the training set were retained, reducing the effective vocab to approximately 1,076 entries. Less frequent subreddits were mapped to an <UNK> token. The embedding dimension was calculated using FastAI heuristic: min(50, vocab_size // 2), resulting in a 50-dimensional vector (Suenaga, 2018). This avoids overfitting to rare communities while still preserving meaningful representations for common ones.

The model was implemented using PyTorch. It is trained using BCEWithLogitsLoss, which is suitable for binary classification as it combines sigmoid activation and binary cross-entropy into a single stable operation. AdamW is used as the optimiser, applying adaptive learning rates with momentum for stable convergence alongside direct weight decay for regularisation. A Cosine Annealing scheduler adjusts the learning rate smoothly across training, stepping at the batch level for finer control. Gradient clipping is applied at each step to prevent exploding gradients.

Due to computational resource constraints, hyperparameter tuning was limited to the learning rate, with three candidate values evaluated: 1e-5, 2e-5, and 5e-5. Other hyperparameters such as dropout rate, batch size, and MLP hidden dimensions were fixed, leaving potential performance gains unexplored.

For each learning rate, a fresh model is initialised, trained up to 30 epochs, and monitored with a custom early stopping class, halting training if no significant improvement is observed over 3 consecutive epochs. The best-performing checkpoint across all learning rates is saved based on validation loss, which was achieved at 2e-5 (Val Loss: 0.4659).

Cross-Evaluation

Lastly, we acknowledge that sarcasm exists in various linguistic and social contexts. The Reddit dataset is one out of many online social settings that sarcasm can occur. Nuances such as age demographic, where redditors tend to be of a younger audience, to the subject matter at hand, subtly shapes the language used in discourse. This causes a potential variance in performance when the model is applied to other corpuses. We have identified two main corpuses to analyse the model's cross compatibility with.

Name of Corpus	Characteristics
Reddit SARC	Large scale sarcasm detection, with over 1.3 million data entries to work with.
IAC-V2	Internet Argument Corpus V2: significantly smaller dataset containing 11,000 annotated quote-response pairs from online political debates => largely political in nature, discourse is more formalised and rhetoric based.
Ghosh Twitter	Sarcasm usually exists in hashtags, and colloquial words or language choices. A corpus doing sentiment analysis in Hindi.

The SARC and IAC-V2 datasets have shown to be structurally similar, as both involved threaded or argumentative online discussions similar to Reddit. Sarcasm is thus naturally occurring through the informal discussions happening on these internet discussion platforms. IAC-V2 gives us the ability to test discourses beyond Reddit while contextual dependencies remain.

The Ghosh dataset has shown to provide more pre-processing steps, though contextually similar. We had to preprocess the Ghosh dataset to take into account hashtags based sarcasm labels, and deal with colloquial languages and capitalised letters. These subtle linguistic variations also allow us to examine the effectiveness of our baseline model, and to improve robustness in subsequent iterations.

III. Model Results

We evaluate our models using standard classification metrics, namely accuracy, precision, recall, F1-score, and ROC-AUC. Accuracy provides an overall measure of correctness, while precision and recall help us understand the trade-off between false positives and false negatives. In particular, F1-score is used as a key metric since it balances precision and recall, which is important for sarcasm detection where both types of errors are meaningful. ROC-AUC is additionally used to evaluate the model's ability to distinguish between classes across different thresholds.

	Test Performed

Model	Accuracy	F1-Score	Precision	Recall	ROC-AUC
Logistic Regression	0.7198	0.7102	0.7354	0.6866	0.7911
Random Forest	0.7127	0.7034	0.7269	0.6814	0.7827
XGBoost	0.7074	0.6968	0.7229	0.6725	0.7814
Multi-Layer Perceptron (with only child comment)	0.7658	0.7675	0.7690	0.7678	0.8492
Multi-Layer Perceptron (with child and parent comment)	0.7747	0.7746	0.7748	0.7747	0.8593
Multi-Layer Perceptron (with child and parent comment and subreddit embedding layer)	0.7774	0.7771	0.7787	0.7774	0.8604

Cross-Evaluation with other datasets

Dataset	Accuracy	F1-Score	Precision	Recall	ROC-AUC
IAC-V2 (GEN)	0.8058	0.8063	0.8075	0.8050	0.8754
IAC-V2 (HYP)	0.7642	0.7651	0.7663	0.7639	0.8512
IAC-V2 (RQ)	0.7203	0.7218	0.7236	0.7199	0.8210
IAC-V2 (Merged)	0.7831	0.7840	0.7852	0.7826	0.8621
Ghosh	0.7619	0.7634	0.7648	0.7609	0.8496

To assess the efficacy and robustness of our model, we have cross-evaluated the sarcasm model with other datasets as mentioned above. The overall assessment is that the performance of our models have suffered due to the dataset. Methodology remained the same. The datasets were treated as per the aforementioned data processing steps, with the cleaned up files utilised being saved in the project file.

On the IAC-V2 dataset, the model achieves its strongest results on the GEN subset, with 0.8058 accuracy, 0.8063 F1-score, and a ROC-AUC of 0.8754. This suggests relatively high reliable categorisation between sarcastic and non-sarcastic instances when lexical cues are direct. Performance remains strong on challenging HYP subset which introduces hyperboles in the texts, attributing to the decrease in accuracy and overall metrics. Moreover, on the pooled dataset, a relatively stable performance is achieved, comparable with the original dataset. For the Ghosh dataset where greater lexical noise and informal variation occurs, the model still performs decently with a comparable score with the IAC-V2 set. This suggests the applicability of our model across various domains.

IV. Discussion of Results & Evaluation

Logistic regression outperforms both tree-based models across all metrics despite being a simpler model. This is structurally expected: TF-IDF produces a 50,000-dimensional sparse feature space, and logistic regression handles this naturally by learning a global weight for every feature simultaneously. Tree-based models, by contrast, must partition this space through sequential splits on individual features. Random Forest further limits each split by considering only a subset of features per node, with the vast majority of features being zero for any given document, resulting in most splits providing negligible signal. For XGBoost, we addressed this directly by applying TruncatedSVD to reduce the sparse TF-IDF matrix to 300 dense dimensions before training — this meaningfully improved performance over the original sparse XGBoost (test F1 0.649 → 0.697), but still fell short of logistic regression. The additional engineered features were similarly insufficient to close the gap.

These bag-of-word approaches have a fundamental ceiling in that the models operate on these words in isolation, with no notion of context, tone, or the relationship between a comment and its parent. A comment containing "obviously" will always look sarcastic to this model regardless of whether it was sincere, and on the other hand, a genuinely sarcastic but lexically neutral reply can never be detected without understanding what it is replying to.

The Multi-Layer Perception Model yielded a Test F1 of 0.7771, a substantial improvement over the logistic baseline's 0.7102. This effectiveness is expected and rationalised by the model's dual-stream architecture:

- **Contextual Embeddings**: Unlike the baseline, the transformer architecture captures long-range dependencies between the parent and child comment**s** through its self-attention mechanism.
- **Subreddit Context**: The addition of the **s**ubreddit embedding layer provides a technical "prior" that helps the model adjust its interpretation of text based on the subreddit's culture.

To further enhance performance beyond the current implementation, several strategies could be explored:

- **Deeper Fine-tuning**: If computational resources allow, unfreezing more layers or the entire DistilBERT or using a larger model like BERT would allow for better domain adaptation.
- **Broader hyperparameter search**: covering dropout, MLP depth, and embedding dimension would likely yield further gains
- **Feature variables**: Evaluating on additional features like word count, author data might help to discover additional patterns.

Another improvement is model interpretability. We consider SHapley Additive exPlanation (SHAP) a future improvement to move the model from a “black box” as it can use its principles from Cooperative Game Theory to give global and local insights. However, it is computationally expensive for our huge datasets. Also, there is an issue technical wise as our model’s embedding layer uses Long Tensor while SHAP requires Floating Tensors. Hence we did not implement it.

Limitations of the model.

One limitation of the MLP model is that the max token length of 256 was chosen based on the median and upper-quartile distribution of combined text lengths. While this captures the majority of examples, longer comments will be truncated, potentially losing sarcasm cues at the end of longer posts.

The project fell short of its broader goal of accurately detecting sarcasm as the model was trained and tested exclusively on a Reddit dataset, which reflects a very specific online subculture and linguistic style. Because Reddit users often rely on platform-specific slang and community-driven context, the results are likely not representative of how sarcasm is expressed in the real world or in other digital environments.

V. Conclusion

In conclusion, this project seeks to use a multimodal approach to find the best classifier that can deal with the detection with sarcasm most effectively. Our results have shown that while logistic regression provide strong and computationally efficient baselines (F1 score $\approx$ 0.71), they are ultimately limited by their inability to capture contextual and relational meanings in sarcasm. Tree-based models like Random Forest and XGBoost perform slightly worse even after dimensionality reduction, reinforcing the limitation of sparse high-dimensional text representations for split based learners. The neural approach conversely consistently outperformed classical methods, with the Multi-Layer Perceptron incorporating both parent-child comment context, and subreddit embeddings achieving the best overall performance. Cross-evaluation also shows reasonable robustness particularly on datasets with clear lexical cues. Performance however declines in more noisy or context dependent settings, highlighting data sensitivity. Ultimately, the results show that incorporating contextual information and hierarchical discourse structure is critical for sarcasm detection, with the MLP-based architecture representing the most effective classifier in this study, albeit still constrained by dataset specificity, token truncation, and limited generalisability beyond Reddit-style language.

V. Appendix: Statement of Independent Work

Declaration of Original Work. By entering our Student ID below, we certify that we completed our assignment independently of all except as stated below.

Signed,

Student 1 -
Student 2 -
Student 3 -
Student 4 -
Student 5 -

Reference List

Khodak, M., Saunshi, N., & Vodrahalli, K. (2017). A large self-annotated corpus for sarcasm. https://arxiv.org/abs/1704.05579

Dubey, P., Dubey, P., & Bokoro, P. N. (2025). Unpacking sarcasm: A contextual and transformer-based approach for improved detection. Computers, 14(3), 95. https://doi.org/10.3390/computers14030095

Hazarika, D., Poria, S., Gorantla, S., Cambria, E., Zimmermann, R., & Mihalcea, R. (2018). CASCADE: Contextual sarcasm detection in online discussion forums. In Proceedings of the 27th International Conference on Computational Linguistics (pp. 1837–1848). Association for Computational Linguistics.

Joshi, A., Sharma, V., & Bhattacharyya, P. (2016). Harnessing context incongruity for sarcasm detection. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (pp. 757–762). Association for Computational Linguistics.

Khodak, M., Saunshi, N., & Vodrahalli, K. (2017). A large self-annotated corpus for sarcasm. arXiv. https://arxiv.org/abs/1704.05579

Pant, K., & Dadu, T. (2020). Sarcasm detection using context separators in online discourse. In Proceedings of the Second Workshop on Figurative Language Processing (pp. 78–83). Association for Computational Linguistics. https://arxiv.org/abs/2006.00850

Potamias, R. A., Siolas, G., & Stafylopatis, A. G. (2020). A transformer-based approach to irony and sarcasm detection. Neural Computing and Applications, 32(23), 17309–17320. https://doi.org/10.1007/s00521-020-05102-3

Šandor, D., & Bagić Babac, M. (2024). Sarcasm detection in online comments using machine learning. Information Discovery and Delivery, 52(2), 213–226. https://doi.org/10.1108/IDD-01-2023-0002

Verma, P., Shukla, N., & Shukla, A. P. (2021). Techniques of sarcasm detection: A review. In *2021 International Conference on Advance Computing and Innovative Technologies*

in Engineering (ICACITE) (pp. 968–972). IEEE. https://doi.org/10.1109/ICACITE51222.2021.9404585

Sajid, H. (2024). DistilBERT: A Distilled Version of BERT.. https://zilliz.com/learn/distilbert-distilled-version-of-bert

Debmanyu Hazarika, Saujanya Poriya, Shruti Gorantala, Eric Cambria, Roger Zimmerman, Rada Mihalcia: CASCADE: Contextual Sarcasm Detection in Online Discussion Forums. https://arxiv.org/abs/1805.06413v1

Suenaga, H. (2018) Deep Learning 2: Part 1 Lesson 4 https://medium.com/@hiromi_suenaga/deep-learning-2-part-1-lesson-4-2048a26d58aa